

Rapport de Stage

Mise en place d'une plateforme DevOps automatisée avec Terraform, Ansible et Docker

Réalisé par : Amen Allah Kochrad

Entreprise : Digital Power Consulting (DPC)

Encadrante : Mme Nibrasse HMIDI

Période : 16 Juin 2025 – 31 Juillet 2025

Établissement : ESPRIT

Année universitaire : 2024 – 2025

Remerciements

Je tiens à exprimer ma sincère gratitude à l'entreprise **Digital Power Consulting (DPC)** pour m'avoir offert l'opportunité d'effectuer ce stage au sein de leur équipe technique.

Je remercie particulièrement **Mme Nibrasse HMIDI**, chef de projet technique, pour son encadrement, ses conseils et son soutien durant toute la période du stage.

Je remercie également l'ensemble de l'équipe pour leur accueil chaleureux et leur aide précieuse qui ont contribué à la réussite de ce stage.

Enfin, je remercie mon établissement **ESPRIT** pour la qualité de la formation qui m'a permis de réaliser ce projet.

Ce stage a été réalisé au sein de l'entreprise **Digital Power Consulting (DPC)** dans le cadre de ma formation en ingénierie informatique.

L'objectif du projet était de mettre en place une **plateforme DevOps automatisée** permettant de déployer plusieurs outils de développement logiciel.

Pour atteindre cet objectif, plusieurs technologies ont été utilisées telles que **Proxmox, Terraform, Ansible et Docker.**

Le projet consiste à automatiser la création de l'infrastructure, la configuration des machines et le déploiement des services DevOps tels que **Jenkins, SonarQube et Nexus.**

Cette solution permet de faciliter la gestion de l'infrastructure et d'améliorer l'efficacité du processus de développement logiciel.

Introduction générale

Avec l'évolution rapide des technologies informatiques, les entreprises cherchent à automatiser leurs infrastructures afin d'améliorer la rapidité et la fiabilité du déploiement des applications.

Le concept **DevOps** est apparu pour répondre à ces besoins en facilitant la collaboration entre les équipes de développement et d'exploitation.

Dans ce contexte, ce stage réalisé au sein de l'entreprise **Digital Power Consulting (DPC)** a pour objectif de mettre en place une **plateforme DevOps automatisée**.

Le projet consiste à utiliser plusieurs outils modernes tels que **Terraform, Ansible et Docker** pour automatiser la création et la configuration de l'infrastructure.

Présentation de l'entreprise

Présentation de Digital Power Consulting

Digital Power Consulting (DPC) est une entreprise spécialisée dans les solutions informatiques et la transformation digitale des entreprises.

Activités principales

- Développement logiciel
- Conseil informatique
- Solutions cloud
- Transformation digitale

Organisation

- Équipe développement
- Équipe infrastructure
- Équipe gestion de projet

Contexte et objectif du stage

- **Pourquoi ce projet ?**
- **Le problème à résoudre ?**

L'objectif principal de ce stage est de concevoir une infrastructure automatisée permettant de déployer facilement une plateforme DevOps.

Cette plateforme doit permettre d'installer automatiquement plusieurs outils nécessaires au développement logiciel.

Technologies utilisées

Proxmox

Plateforme de virtualisation utilisée pour créer des containers LXC.

Terraform

Outil d'Infrastructure as Code permettant de créer automatiquement les machines.

Ansible

Outil d'automatisation permettant de configurer les machines à distance.

Docker

Plateforme de containerisation utilisée pour déployer les applications.

Jenkins

Outil d'intégration continue.

SonarQube

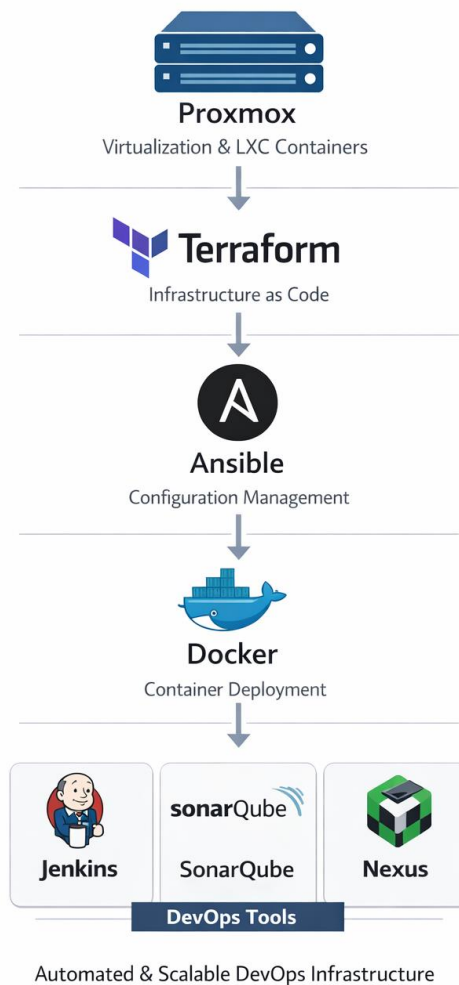
Outil d'analyse de qualité du code.

Nexus

Repository d'artefacts.

Réalisation du projet

1.1: Architecture du projet :



1.2 : Installation de Proxmox

La première étape du projet consiste à installer la plateforme de virtualisation **Proxmox VE** sur une machine physique.

Proxmox est une solution open source permettant de gérer facilement des machines virtuelles et des containers.

Installation

Après avoir téléchargé l'image ISO officielle de Proxmox, l'installation est réalisée sur un ordinateur dédié.

Les principales étapes de l'installation sont :

1. Démarrage de la machine avec l'image ISO Proxmox
2. Configuration du disque de stockage
3. Configuration du réseau
4. Création du compte administrateur

Une fois l'installation terminée, l'interface web de Proxmox devient accessible via un navigateur :

[https://IP DU SERVEUR:8006](https://IP_DU_SERVEUR:8006)

Cette interface permet de gérer :

- les containers
- les machines virtuelles
- le stockage
- le réseau

Importation d'un template Ubuntu

Afin de créer des containers rapidement, un **template Ubuntu 22.04** a été téléchargé depuis la bibliothèque de templates de Proxmox.

Ce template sera utilisé pour créer les différents containers nécessaires au projet.

Les containers suivants ont été créés :

Container	Rôle
ansible	machine de configuration
jenkins	serveur Jenkins
sonarqube	serveur SonarQube
nexus	repository Nexus

1.3 : Création des containers avec Terraform :

Afin d'automatiser la création des containers, l'outil **Terraform** a été utilisé.

Terraform est un outil d'**Infrastructure as Code (IaC)** permettant de définir l'infrastructure sous forme de code.

Fichier de configuration Terraform

Le fichier principal utilisé est :

main.tf

Ce fichier contient la définition des containers LXC.

Exemple de configuration réseau :

```
network {  
    name     = "eth0"  
    bridge   = "vmbr0"  
    ip       = "192.168.0.200/24"  
    gw       = "192.168.0.1"  
}
```

Chaque container possède une **adresse IP statique** afin de faciliter la communication entre les machines.

Initialisation du projet Terraform

Avant d'exécuter Terraform, il est nécessaire d'initialiser le projet :

terraform init

Cette commande permet de télécharger les plugins nécessaires.

Création des containers

Ensuite, la commande suivante est utilisée :

terraform apply

Terraform analyse le fichier de configuration puis crée automatiquement les containers dans Proxmox.

Cette approche permet de reproduire facilement l'infrastructure sur un autre environnement.

1.4 Configuration SSH :

Une fois les containers créés, il est nécessaire de configurer l'accès **SSH** afin de permettre la communication entre les machines.

SSH permet de se connecter à distance à un serveur en ligne de commande.

Test de connexion

Depuis la machine locale, les connexions SSH ont été testées avec la commande suivante :

```
ssh root@192.168.0.200
```

Chaque container possède une adresse IP spécifique :

Container	IP
Jenkins	192.168.0.200
SonarQube	192.168.0.201
Nexus	192.168.0.202

1.5 Installation et configuration d'Ansible :

Afin d'automatiser la configuration des machines, un container dédié a été utilisé pour installer **Ansible**.

Ansible est un outil d'automatisation permettant de gérer plusieurs serveurs simultanément.

Installation d'Ansible

Sur le container **ansible**, les commandes suivantes ont été exécutées :

apt update

apt install ansible -y

Configuration de l'inventaire

Ansible utilise un fichier appelé **inventory** qui contient la liste des machines à gérer.

Ce fichier se trouve dans :

/etc/ansible/hosts

Exemple de configuration :

[containers]

```
jenkins ansible_host=192.168.0.200 ansible_user=root  
ansible_password=jenkins2025
```

```
sonarqube ansible_host=192.168.0.201 ansible_user=root  
ansible_password=sonarqube2025
```

```
nexus ansible_host=192.168.0.202 ansible_user=root  
ansible_password=nexus2025
```

Test de connectivité

Pour vérifier la communication entre Ansible et les machines, la commande suivante est utilisée :

ansible all -m ping

Si la configuration est correcte, la réponse suivante apparaît :

pong

1.6 Installation de Docker avec un Playbook Ansible :

Afin de déployer les services DevOps, il est nécessaire d'installer **Docker** sur chaque container.

Pour automatiser cette installation, un **playbook Ansible** a été créé.

Un playbook est un fichier YAML contenant une liste de tâches à exécuter.

Exemple de playbook

Fichier :

install_docker.yml

Exemple de tâches :

- name: Installer Docker
 - hosts: containers
 - become: yes
 - tasks:
 - name: Installer les dépendances
 - apt:
 - name: apt-transport-https
 - state: present
 - name: Installer Docker
 - apt:
 - name: docker-ce
 - state: present

Exécution du playbook

La commande suivante permet d'exécuter le playbook :

ansible-playbook install_docker.yml

1.7 Déploiement des services Jenkins, SonarQube et Nexus :

Une fois Docker installé, les services DevOps peuvent être déployés sous forme de containers Docker.

Cette étape a été automatisée à l'aide d'un deuxième playbook appelé :
`deploy_services.yml`

Déploiement Jenkins

- name: Lancer Jenkins
 - docker_container:
 - name: jenkins
 - image: jenkins/jenkins:lts
 - ports:
 - "8080:8080"

Déploiement SonarQube

- name: Lancer SonarQube
 - docker_container:
 - name: sonarqube
 - image: sonarqube:latest
 - ports:
 - "9000:9000"

Déploiement Nexus

- name: Lancer Nexus
 - docker_container:
 - name: nexus

```
image: sonatype/nexus3
ports:
  - "8081:8081"
```

Accès aux services :

Service	URL
Jenkins	http://192.168.0.200:8080
SonarQube	http://192.168.0.201:9000
Nexus	http://192.168.0.202:8081

Problèmes rencontrés et solutions:

1.1 :Problème de synchronisation de l'heure système :

Lors de l'exécution du playbook Ansible pour installer Docker, une erreur est apparue lors de la mise à jour des paquets avec **APT**.

Message d'erreur: **Release file is not valid yet**

1.2 : Problème de clé GPG Docker :

Lors de l'ajout du dépôt officiel de Docker, une erreur liée à la clé GPG est apparue.

Message d'erreur : `Key is stored in legacy trusted.gpg keyring`

1.3 : Problème de connexion SSH avec Ansible :

Au début du projet, Ansible ne parvenait pas à se connecter aux containers.

Message d'erreur : `Permission denied (publickey,password)`

1.4 : Problème de communication réseau entre containers

1.5 : Problème de téléchargement des images Docker

Conclusion :

Ce stage m'a permis d'acquérir une expérience pratique dans la mise en place d'une infrastructure DevOps automatisée.

J'ai pu découvrir plusieurs outils modernes tels que **Terraform**, **Ansible** et **Docker** et comprendre leur rôle dans l'automatisation des infrastructures informatiques.

Ce projet m'a également permis de développer mes compétences techniques et ma capacité à résoudre des problèmes techniques.

Présentation de Digital Power Consulting.....	1
Activités principales	1
Organisation.....	1
Proxmox.....	1
Terraform	1
Ansible.....	1
Docker	1
Jenkins	1
SonarQube.....	1
Nexus.....	1
1.1: Architecture du projet :.....	1
Installation	1
Importation d'un template Ubuntu	1
Fichier de configuration Terraform.....	1
Initialisation du projet Terraform.....	1
Création des containers	1
Test de connexion	1
Installation d'Ansible.....	1
Configuration de l'inventaire	1
Test de connectivité	1
Exemple de playbook.....	1
Exécution du playbook.....	1
1.7 Déploiement des services Jenkins, SonarQube et Nexus :	1
Déploiement Jenkins	1
Déploiement SonarQube	1
Déploiement Nexus.....	1
Accès aux services :	1
Message d'erreur : Key is stored in legacy trusted.gpg keyring.....	1
Message d'erreur : Permission denied (publickey,password)	1

